arXiv:2608.11350v2 [cs.CL] 10 Sep 2026

自进化具身智能体通过技能-套件进化

王培东[1], [2], [*], [§] **马志明**[1], [*] **常颖**[2], [*], [§] **罗旭方**[2], [†] **张一群**[1] **王志涵**[1] **杨晓翠**[1] **冯时**[1], [‡] **杨雨晴**[2] **李东升**[2]

[1]东北大学[2]微软研究院

Abstract

具身智能体越来越多地围绕基础模型构建，其中性能不仅取决于模型权重，还取决于围绕模型的技能、上下文、动作接口和执行套件。虽然监督微调和强化学习可以使智能体适应新环境，但它们需要额外的数据、奖励和训练运行；同时，许多无训练代码为中心的方法依赖于可编程机器人API，这些API在固定接口设置中可能不可用。我们提出了SHAPER，一个用于无训练具身自适应的自进化框架，该框架冻结模型参数，并通过目标环境 rollout 通过进化可重用技能和上下文代码套件来改进非参数智能体系统。在SHAPER中，相同的冻结模型可以作为规划器和优化器，而无需参数更新来完善其外部技能和上下文代码套件。我们在VLABench 和 ESI-Bench 上评估了SHAPER，涵盖了具有不同低级动作接口的具身智能体，并与纯执行、监督微调和测试时缩放基线（如无验证器选择和投票）进行了比较。我们的结果表明，当模型训练昂贵、不可用或不受欢迎时，技能-套件优化是自进化具身智能体的实用途径。

1 简介

具身人工智能正越来越多地从孤立的策略模型转向智能体系统。近期的视觉-语言-行动模型和多模态基础模型表明，大型预训练模型能够感知场景、解释指令、对物体和空间关系进行推理，并在模拟或物理环境中行动 Kim 等人 [2025], Black 等人 [2024, 2025], 物理智能 等人 [2026], NVIDIA 等人 [2025], Gemini机器人团队 等人 [2025], Yuan 等人 [2026]。与此同时，智能体脚手架、智能体-计算机接口、套件优化和机器人技能发现方面的工作表明，智能体的行为不仅仅由模型决定 Yang 等人.[2024], Chen 等人 [2026], Pan 等人. [2026a], Lee 等人. [2026],Lou 等人 [2026], Ju 等人 [2026],Lu 等人 [2026]。一个部署的具身智能体受到模型外部组件的影响，包括可重用技能、上下文构建、动作接口、输出解析和执行包装器。这些非参数选择在具身环境中尤其重要，其中智能体必须在有限的视觉反馈下通过受限的动作接口行动。

一种常见的适应具身智能体的方法仍然是更新模型参数。监督微调（SFT）和强化学习（RL）后训练可以提高具身智能体的性能。

同等贡献
通讯作者：luoxufang@outlook.com。
通讯作者：fengshi@cse.neu.edu.cn。Peidong Wang 和 Ying Chang 在微软亚洲研究院实习期间完成的工作。

预印本。

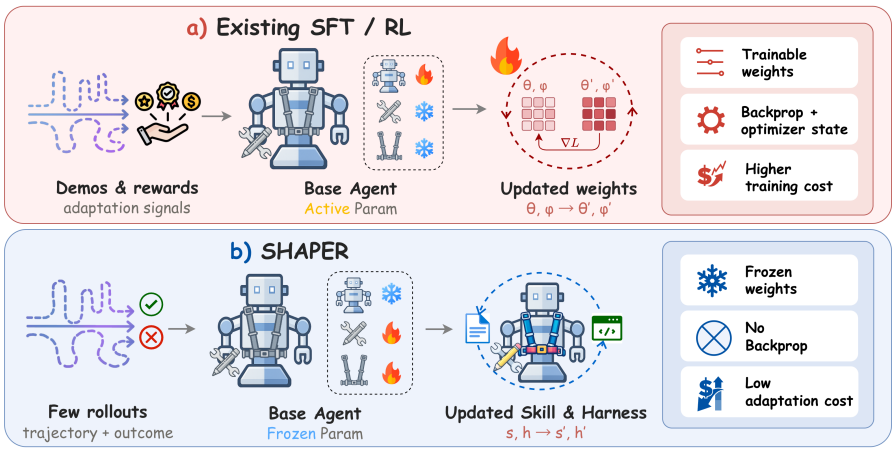


图1：SHAPER通过从少量 rollout 中进化技能和 harness 来适应冻结的具身智能体。

mance Open X-Embodiment Collaboration et al. [2023], Khazatsky 等人 [2024], Guo 等人 [2025],Lu 等人 [2025], Pan 等人 [2026b], 物理智能 等人. [2025],但需要访问模型权重、任务特定演示或奖励，并进行额外的优化。这些要求在模型权重不可用或交互数据稀缺时限制了适应性。这促使了一个补充问题：如何在不更新参数的情况下将一个强冻结的具身智能体适应到目标环境?

Train-free embodied adaptation offers a different path. Recent agentic robotics systems show that procedural knowledge can live outside the model: language models can write robot programs, revise failed executions, and accumulate reusable skills without directly changing neural weights Ahn et al. [2022], Liang et al. [2023], Huang et al. [2023], Wang et al. [2023], Ju et al. [2026], Lu et al. [2026]. However, much of this progress treats executable code as the action substrate. The agent improves by generating or repairing programs that call robot-specific APIs Liang et al. [2023], Lu et al. [2026], which is powerful when such APIs, debugging hooks, and execution monitors are available. In many embodied environments, these interfaces may be absent, incomplete, or difficult to expose, and the agent may instead have to act through a fixed or restricted action space. This leaves open how to systematically adapt embodied agents by improving the planner-side procedural guidance and context that shape behavior, rather than by synthesizing low-level control programs.

如图1所示，我们提出了SHAPER，一个用于无训练具身自适应的自进化框架。关键区别在于自适应的内容。SFT和RL通过反向传播将演示或奖励转化为模型权重的更新；而SHAPER则将一小部分目标环境轨迹和结果转化为可重用技能和上下文代码harness的修订，同时冻结规划和执行器的权重。这里，技能是一段程序指导，可以写成自然语言、动作配方、故障恢复规则或任务分解策略。上下文代码harness决定了向规划器呈现哪些观察、先行动作、执行结果和反馈，以及如何组织它们。我们使用相同的冻结VLM检查点扮演两个不同的角色。在环境交互期间，它作为高级规划器。在进化期间，它被单独提示作为工件优化器，根据总结的 rollout 反馈修订文本技能和上下文代码harness。这两个角色共享模型权重，但使用不同的提示和输入。因此，代理在保持其模型参数固定的情况下，进化其外部技能和上下文构建。

我们在具有不同底层动作接口的具身环境中评估 SHAPER。在 VLABench 中，张等人 [2024], 将上层 VLM 代理通过一个 VLA actor 连接到环境，该 actor 作为低层执行工具。在 ESI-Bench 中，黄等人 [2026], 相同的 agentic 公式通过基准的 environmentaction 接口起作用。在这些设置中，我们与纯执行、同数据 SFT 和测试时缩放 (TTS) 基线进行比较，例如无验证器选择和轨迹投票 Jang 等人 [2025], Lin 等人 [2025]。我们的结果旨在表明，当参数更新昂贵、不可用或不受欢迎时，进化技能和套件是一种改进具身智能体的实用方法。

本文做出了三项贡献：(1) 我们将无训练具身自适应公式化为具身智能体系统的非参数优化，其中可重用技能和上下文代码套件围绕冻结模型进行优化；(2) 我们引入 SHAPER，一个自进化框架，通过目标环境 rollout 改进具身智能体，通过进化文本技能和上下文代码套件，而无需更新模型参数；(3) 我们在具有不同动作接口的具身环境中评估 SHAPER，表明技能-套件优化可以改进具身性能，并为微调和采样密集型基线提供有竞争力的替代方案。

2 相关工作

具身基础模型和参数更新适应。 近期的具身基础模型将大规模视觉和语言预训练与机器人控制相结合。RT-2、OpenVLA、Octo和 π_0 建立了通才型VLA策略，而 $\pi_{0.5}$、 $\pi_{0.7}$、GR00T N1、Gemini Robotics和Qwen-RobotManip扩展了开放世界泛化、可引导性、人形控制和具身推理能力。Zitkovich等人 [2023], Kim等人 [2025], Octo模型团队等人 [2024], Black等人 [2024, 2025], 物理智能等人 [2026], NVIDIA等人 [2025], Gemini机器人团队等人 [2025],Yuan等人 [2026]。通过监督学习或强化学习进行的大型机器人数据集和参数更新适应进一步扩展了他们的能力。开放X-具身化协作等人[2023], Khazatsky等人 [2024], Guo等人 [2025], Lu等人 [2025], Pan等人 [2026b], Shi等人[2026], 物理智能等人 [2025]。这些方法仍然依赖于额外的数据、奖励或纠正以及模型更新；我们通过外部技能和harness工件来适应冻结的代理。

自进化的代理。 近期工作通过编辑模型外部的工件来适应冻结的代理。Skil-Opt 通过有界编辑和验证门控的 rollout 优化一个持久的自然语言技能，同时保持执行 harness 固定 Yang 等人 [2026]。EmbodiSkill 通过区分有缺陷的技能内容与执行失误，将技能进化带到具身环境中，但同样更新的是技能而非上下文 harness Ju 等人. [2026]。AutoHarness 则从文本游戏环境中的环境反馈中合成一个代码 harness，甚至是一个完整的代码策略，而无需协同优化可重用的程序指导 Lou 等人. [2026]。AgentSpec 将具身框架表示为带类型的可互换组合，用于受控分析而非 rollout 驱动的工件优化 Chen 等人. [2026]。相比之下，SHAPER 从环境交互中进化具身代理的可重用技能和上下文代码 harness，同时保持所有模型参数冻结。

Train-free adaptation for embodied agents. Train-free embodied systems compose frozen models with affordances, executable robot APIs, spatial value maps, or reusable skills Ahn et al. [2022], Liang et al. [2023], Huang et al. [2023], Ju et al. [2026], Lu et al. [2026]. Code- and API-centric approaches, however, depend on the programmable interfaces, perception modules, and execution monitors available on each platform. For VLA policies, vision-language steering, latent prompt optimization, recurrent-depth reasoning, verifier guidance, and test-time sampling or voting improve action execution without retraining Liu et al. [2026], Zhang et al. [2026], Tur et al. [2026], Singhi et al. [2026], Kwok et al. [2025], Jang et al. [2025], Lin et al. [2025], Yang et al. [2025]. SHAPER instead targets fixed action interfaces and uses rollout feedback to jointly evolve reusable high-level skills and harness context.

3 方法

3.1 无训练代理分解

现代具身智能体很少使用单一的神经策略独立运行。如图2A所示，我们将具身智能体分解为四个系统组件：一个VLM规划器 M_θ、一个执行器 $\mathcal{A}_\phi$、一个可重用技能 s和一个上下文代码 harness h。执行器可能是一个VLA actor或一个环境动作API的包装器。技能是规划器提示中的文本程序指导，涵盖场景检查、任务分解、子目标选择、恢复和终止。harness是一个上下文构建器——实现为一个Python函数，例如build_context，用于选择和

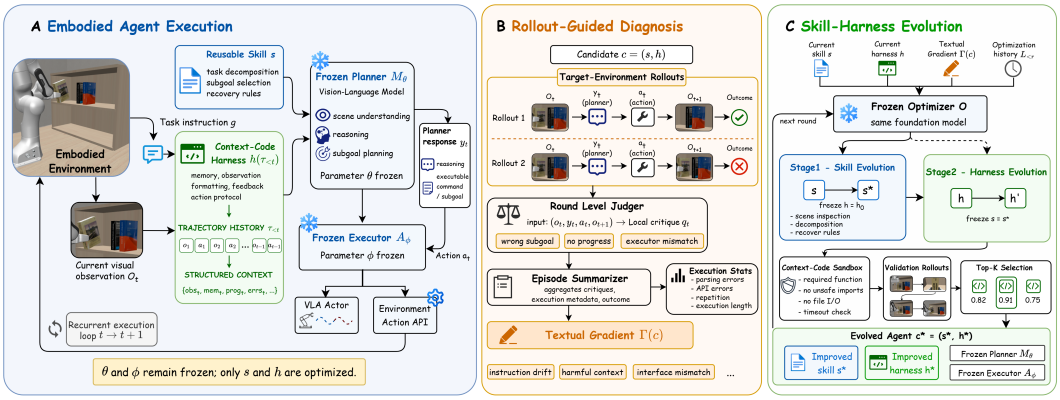


图2：SHAPER 概述。(A) 文本技能和上下文代码 harness 条件化一个冻结的规划器，该规划器连接到一个冻结的执行器。(B) 在回合和回合级别诊断目标环境 rollout 以形成文本梯度。(C) 冻结的优化器进化技能然后 harness，并进行沙盒验证和 top-K 选择。

为规划器格式化轨迹历史。动作接口、输出解析器和执行器保持固定。

对于任务指令 g、当前观察 o_t 和轨迹前缀 $\tau_{<t}$，代理的操作为

$$\begin{aligned} y_t &\sim M_\theta(\cdot \mid g, o_t, s, h(\tau_{<t})), \\ a_t &= \mathcal{A}_\phi(y_t, o_t), \qquad \theta, \phi \text{ frozen; } s, h \text{ optimized.} \end{aligned} \tag{1}$$

这里 y_t 包含规划器的推理和接口级命令，而 $\mathcal{A}_\phi$ 将此命令映射到执行动作 a_t。无训练适应冻结参数化组件 (θ, ϕ) 并仅优化模型外部对 $c = (s, h)$。

设 $\mathcal{E}$ 为一个具有任务分布 $\mathcal{D}$的部分可观察的具身环境。候选 c 在任务 $g \sim \mathcal{D}$ 上的 rollout 包含 T 执行动作和 $T+1$ 观察：

$$\tau^c(g) = \big(o_0, y_0, a_0, \ldots, o_{T-1}, y_{T-1}, a_{T-1}, o_T, R\big),$$

其中 $R \in [0, 1]$ 是任务级奖励或成功信号。SHAPER 寻求最大化预期 rollout 性能的工件：

$$\begin{aligned} (s^\star, h^\star) &= \arg\max_{s \in \mathcal{S},\, h \in \mathcal{H}} J(s, h), \\ J(s, h) &= \mathbb{E}_{g \sim \mathcal{D}} \left[R\big(\tau^{(s,h)}(g)\big) \right]. \end{aligned} \tag{2}$$

在实践中，J 是在有限的训练和验证集上进行估计的。

3.2 Rollout-GuidedTextual Diagnosis

在优化轮 r，每个候选都在一个小型的目标环境批处理 $\mathcal{D}_{\text{tr}}^r$ 上进行评估。具身 rollout 可以包含许多前后图像，使得直接批处理级别的诊断上下文密集且掩盖了导致失败的哪个动作。图 2B 显示了我们的分层替代方案。首先，轮级判别器比较每个规划器响应和动作与执行前后的观察结果：

$$q_t^c(g) = \text{Judge}\big(o_t, y_t, a_t, o_{t+1}\big), \qquad t = 0, \ldots, T-1.$$

这些局部评论可以识别诸如不正确的子目标、缺乏进展或执行器不匹配等效果，而无需将优化器暴露于完整的视觉轨迹。

我们随后构建紧凑的非视觉元数据 $m^c(g)$，包括已发布的命令、执行长度和运行时错误。一个剧情总结器将此元数据与终端 harness 上下文、轮次评论和最终结果相结合。批次的总结和聚合执行

统计数据形成文本梯度：

$$\begin{aligned} e^c(g) = \mathrm{Summarize}\big(m^c(g), h(\tau_{<T}), \{q_t^c(g)\}_{t=0}^{T-1}, \\ R(\tau^c(g))\big), \\ \Gamma(c) = \mathrm{Concat}\left(\{e^c(g) : g \in \mathcal{D}_{\mathrm{tr}}^r\}, \mathrm{Stats}(c)\right). \end{aligned} \tag{3}$$

由此产生的 $\Gamma(c)$ 暴露了可操作的跨轮模式，例如无益重复、指令漂移、缺失进度跟踪、有害上下文以及解析或API错误。

3.3 两阶段技能- harness 进化

从 $c_0 = (s_0, h_0)$ 开始，SHAPER 维护一个候选者束 $\mathcal{B}_r$。如图2C所示，规划器使用的相同冻结的基础模型在单独的优化器提示下被调用。基于当前的候选者、其文本梯度和优化历史 $\mathcal{L}_{<r}$，优化器提出

$$c' = (s', h') \sim \mathcal{O}\left(c, \Gamma(c), \mathcal{L}_{<r}\right), \tag{4}$$

其中 $\mathcal{O}$ 表示此优化器角色。我们使用两阶段调度：技能进化首先使用固定的种子 harness 更新 s，然后 harness 进化使用固定的选定技能更新 h：

$$\begin{aligned} s' \sim \mathcal{O}_s(s, h_0, \Gamma, \mathcal{L}), \quad h' = h_0, \\ h' \sim \mathcal{O}_h(s^\star, h, \Gamma, \mathcal{L}), \quad s' = s^\star. \end{aligned} \tag{5}$$

生成的 harness 在 rollout 前被沙盒化：它们必须定义所需上下文函数，避免导入、文件 I/O 和动态执行，并在超时内完成。无效候选者被拒绝，并返回其验证错误作为优化器反馈。有效候选者在固定的验证集 $\mathcal{D}_{\mathrm{val}}$ 上被评估，束保留了前-K 的在位者和提议：

$$\begin{aligned} \widehat{J}_{\mathrm{val}}(c) = \frac{1}{|\mathcal{D}_{\mathrm{val}}|} \sum_{g \in \mathcal{D}_{\mathrm{val}}} R\big(\tau^c(g)\big), \\ \mathcal{B}_{r+1} = \mathrm{TopK}\left(\mathcal{B}_r \cup \mathcal{C}_r; \widehat{J}_{\mathrm{val}}, K\right), \end{aligned} \tag{6}$$

where $\mathcal{C}_r$ is the set of newly proposed candidates. The final output is the highest-scoring candidate $c^\star = (s^\star, h^\star)$ encountered across rounds. It retains the frozen planner and executor while changing only the planner's textual guidance and constructed context.

4 实验

我们评估技能-套件进化是否能够提升冻结的上层VLM规划器在两个具有不同执行接口的具身环境中的表现。在VLABench中，规划器将文本子目标委托给冻结的VLA演员；在ESI-Bench中，它通过基准测试的固定动作API进行交互。在两个环境中，SHAPER都保持所有规划器和执行器参数不变，仅改变外部技能和套件工件。

4.1 实验设置

Benchmarks. VLABench Zhang et al. [2024] is a language-conditioned manipulation benchmark targeting implicit language understanding, common-sense and world-knowledge transfer, and long-horizon reasoning, all of which are underrepresented in the earlier manipulation benchmarks considered. Its instructions often imply rather than explicitly state goals, and task instances vary object categories to test generalization.

表 1：VLABench 划分设计。

	目标类别	
任务表单	Seen	未见
语义理解	**C1**	**C2**
常识	**C3**	**C4**

我们关注语义理解，它测试从微妙语言中恢复预期目标的能力，以及常识与世界知识，它要求在观察到的场景中扎根先验知识。

方法	C1	C2	C3	C4	总体
VLA级别基线			无上层规划器		
直接VLA	26.0	9.5	45.0	12.5	23.25
SFT（相同数据）	32.5	13.0	38.0	12.5	24.00
Test-time scaling			推理时聚合		
VLA + MG-Select	25.0	12.0	29.5	19.0	21.38
VLA + VOTE	26.0	12.5	33.0	16.5	22.00
种子代理 + MG-Select	35.0	15.0	40.5	7.0	24.38
种子代理 + VOTE	38.0	13.0	34.0	9.0	23.50
无训练代理自适应			冻结的规划器与执行器		
种子代理	40.0	20.0	40.0	13.0	28.25
技能进化	42.0	23.5	**50.0**	18.5	33.50
harness 进化	42.0	21.0	46.0	13.0	30.50
SHAPER	**42.5**	**26.0**	**50.0**	**19.5**	**34.50**

表 2：VLABench 在四个 200-集划分上的成功率（%）；最佳结果加粗。

如表1所示，C1–C4是我们的自定义保留评估拆分，而非官方VLABench拆分。C1完全在领域内；C2使用未见目标类别；C3将任务表单更改为常识与世界知识；而C4则同时改变两者。每个单元格包含五个任务系列和每个系列40个保留情节，给出每个单元格200个情节和总体800个情节。

ESI-Bench Hong 等人 [2026] 在一个跨越 10 个任务类别的封闭感知-行动循环中评估具身空间智能。与被动的空间问答不同，代理必须通过感知、移动和操作主动获取初始视图不可用的证据。我们在一个包含 231 个问题的子集上评估，该子集按官方基准的类别比例进行抽样，并报告总体和类别级别的准确率。

模型和执行接口。所有代理系统变体在评估时使用 Qwen3.6-27B 作为冻结的上级规划器。在 VLABench 上，执行器是官方 VLABench π_0 检查点，在基准的十个原语任务类别上进行微调，并通过 OpenPI 提供。它接收一个文本子目标，并产生五动作块的低级控制。上级规划器为每个子目标指定环境步预算，因此一个规划轮次可以执行多个块。一个回合最多允许 10 个规划轮次和 400 个低级环境步。在 ESI-Bench 上，相同的规划器模型通过基准的固定交互 API 执行，最多 30 步。

优化协议。对于VLABench，我们在15个训练集中进化工件，并在固定的24集验证集上选择候选者；这两者都与800个保留的评估集互斥。对于ESI-Bench，我们使用10个训练问题和10个验证问题，所有这些问题都与231个问题的评估子集互斥。对于这两个基准测试，进化使用宽度为3、分支因子为2的四个束搜索轮，文本反馈由4个rollout的小批量形成。我们首先优化技能，然后在保持进化技能固定的情况下优化harness。

对比方法。我们根据 VLABench 基线修改的内容进行组织。执行器级基线包括直接 π_0 执行和在同一 15 个训练集上对相同 actor 架构进行 SFT。测试时缩放包括 MG-Select Jang 等人 [2025]和 VOTE Lin 等人 [2025] ，它们直接应用于 VLA actor 或在种子规划器之上应用。代理系统适应包括种子代理、技能进化、harness 进化和完整技能-harness 进化。在 ESI-Bench 上，我们比较种子、进化技能和完整变体，并仅将基准论文中的 GPT-5 被动单视图 (PS) 作为外部公开参考展示。

4.2 VLABench 结果

主要结果。 表2显示，种子代理达到了28.25%，提高了直接VLA执行5.00个百分点，提高了同数据SFT 4.25个百分点，且未改变模型。完整技能-harness 进化进一步将成功率提高到34.50%，比种子高6.25个百分点，比

类别	n	种子代理	进化技能	SHAPER	*GPT-5 PS*†
物理结构	21	14.3	38.1	38.1	51.4
物理动力学	10	40.0	40.0	40.0	46.1
镜面反射	20	10.0	20.0	60.0	53.0
感知基础	49	53.1	59.2	69.4	24.9
度量比较	21	33.3	47.6	47.6	48.2
枚举感知	18	16.7	33.3	27.8	10.7
空间关系	51	27.5	37.3	54.9	31.0
认知地图	17	29.4	23.5	23.5	60.4
时间理解	19	47.4	47.4	47.4	40.6
动作序列	5	40.0	40.0	20.0	36.4
微观平均	231	32.5	41.1	**49.8**	–
宏观平均	10个类别	31.2	38.6	**42.9**	40.3

表 3：ESI-Bench 准确率 (%) 在官方比例 231 题子集上。 n 表示评估的题目数量。GPT-5 PS† 是在完整评估集上发布的被动单视图结果。

SFT高10.50个百分点。相比之下，VLA级MG-Select和VOTE达到了21.38%和22.00%，均低于直接VLA执行，而它们的种子代理版本分别达到了24.38%和23.50%，低于未缩放的种子代理。因此，工件进化的改进在这个设置中没有被额外的推理时采样或聚合重现。

跨偏移泛化。 四个分割分别变化目标类别和任务表单。完整 SHAPER 在领域内 C1 分割上达到 42.5%，而种子为 40.0%。其描述性收益在分布偏移下更大： +6.0 分对于未见目标 (C2)， +10.0 分对于未见任务表单且目标已见 (C3)，以及 +6.5 分当两者都未见 (C4)。它在 C1、C2 和 C4 上获得最佳结果，并在 C3 上与技能进化并列 50.0%。因此，最大收益发生在将进化工件从语义理解转移到常识与世界知识任务表单时。

工件消融。 仅技能、仅适配器和完全进化分别得分为 33.50%、30.50% 和 34.50%，而种子为 28.25%。技能进化解释了与种子最大的描述性差异 (+5.25 分)，而仅适配器为 +2.25 分。在技能进化后添加进化的适配器使总体得分变化 +1.00 分；分割差异为 +0.5 在 C1，+2.5 在 C2，0.0 在 C3，以及 +1.0 在 C4。效果不是可加的：例如，仅适配器将 C3 从 40.0% 提高到 46.0%，但完整和仅技能配置都获得 50.0%。因此，我们将其解释为完整的代理配置，而不是独立工件效应的估计。

演员感知恢复。图3比较种子代理和技能进化在select_book/ep_024上具有相同上下文构建时的表现。种子技能要求规划器在生成子任务前检查任务、相机观察和执行历史。它不指定VLA actor的命令分布偏好、区分选择与检索或放置、强制实体名称精确匹配，也不要求显式进度检查。因此，在目标书籍部分从书架上被拉出后，规划器将持续的可见性视为证据，认为应重复相同的检索命令。该场景在三个规划轮次后达到400步限制，奖励为0.5。

进化技能将规划器转变为演员感知命令适配器。它将任务意图路由到短规范命令，每轮发出一个原语，保留完整的书名，并排除由VLA处理的抓取方向或接近几何等低级细节。它要求规划器将之前的尝试分类为成功、失败、部分成功或停滞，并在重复执行没有进展时更改规范命令。在同一回合中，规划器检测到剩余的货架重叠，从原始检索命令切换到拾取命令，后来认识到书已从货架上清除。它在使用297个环境步长的三个规划轮次中获得了1.0的奖励。

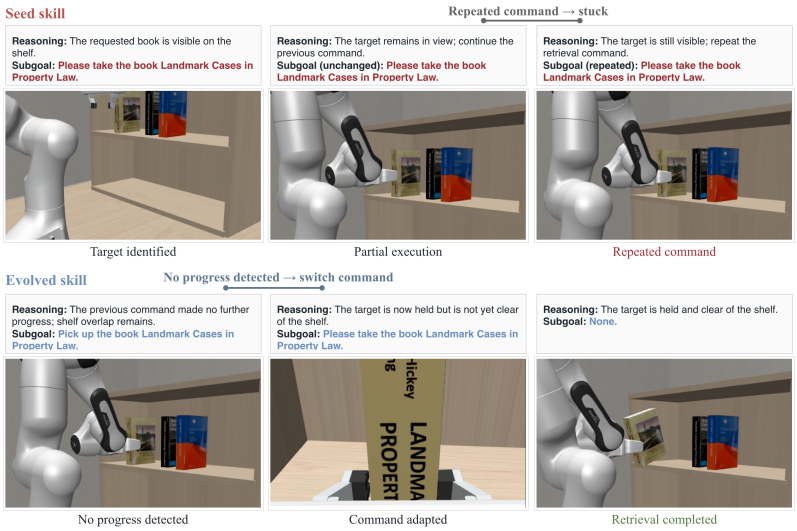


Figure 3: Controlled VLABench case isolating skill evolution for the instruction "Fetch the Landmark Cases in Property Law book for tomorrow's class." The planner, VLA executor, and raw harness are fixed. The seed skill repeats a retrieval command after partial execution, whereas the evolved skill detects no progress, changes the command, and completes retrieval. Text annotations summarize observable decisions rather than quote hidden reasoning.

4.3 ESI-Bench 结果

主要结果。表3显示，完全SHAPER达到49.8%微观和42.9%宏观准确率，而种子代理为32.5%/31.2%，技能进化为41.1%/38.6%。因此，完全进化在微观上比种子高17.3个点，宏观上高11.7个点，比仅技能高8.7个微观点和4.3个宏观点。其宏观得分在数值上高于已发布的GPT-5 PS参考（42.9% vs. 40.3%），冻结的27B规划器在不更新参数的情况下处于类似的综合范围内。这仍然是一个外部而非配对比较：GPT-5 PS使用完整的被动单视图设置，而我们的方法使用固定的231个问题的子集。

类别级行为。 技能进化在十个类别中比种子改进了六个，包括物理结构、镜面反射、感知基础、度量比较、枚举感知和空间关系。添加进化的harness在镜面反射（20.0%到60.0%）、感知基础（59.2%到69.4%）和空间关系（37.3%到54.9%）上产生了最清晰的积极差异。五个类别保持不变，而枚举感知从33.3%下降到27.8%，动作序列从40.0%下降到20.0%。这些结果是描述性的，应结合样本数量进行解释；例如，动作序列只包含五个问题。以下反思案例说明了最大积极差异背后的上下文行为。

Evidence-aware visual memory. Figure 4 compares Skill Evolution with full SHAPER under the same evolved skill. The raw harness concatenates the complete textual action and reasoning history while retaining only the five most recent RGB observations. This recency-based window gives repeated or blank views the same status as earlier task-critical evidence, and the uncompressed text may preserve the model's own mistaken observations. In the shown episode, the agent sees one target jar in the mirror at step 1, but that visual anchor is no longer available when the three real candidates are later observed. It spends all 30 steps oscillating between views, taking 15 right turns, 12 left turns, and 3 forward moves before answering "not sure."

进化后的harness用稀疏证据感知内存替换了这个窗口。它从完整历史中选择最多三个信息量丰富且视觉多样化的关键帧，将每个RGB观测值与一个确定性焦点裁剪配对，并压缩动作路径和最近的执行记录。对于当前视图，它还额外提供选定的水平带和重叠瓦片裁剪。所有裁剪都是使用对比度、边缘密度和清晰度从官方RGB像素计算的，没有目标框

图 4：ESI-Bench 案例隔离 harness 进化，指令为“您给定一个带有镜子与三个面条_罐子的图像。基于镜面反射，同图中哪三个面条_罐子对应反射的：左侧、中间还是右侧物体？”规划器、动作接口与进化技能固定。原始 harness 丢弃早期反射证据，而进化 harness 保留反射目标并针对跨视图匹配进行确定性焦点裁剪，正确选择中间候选。

或模拟器元数据。对于反思任务，构建的上下文明确保留一个反射目标的帧和另一个真实候选者的帧用于外观匹配。在行动之前，规划器报告结构化的 VIEW、TARGETS、EVIDENCE、READY 和 GAP 字段，而 harness 标记低信息重复和逆动作循环。在图 4 中，步骤 1 的反思在步骤 7 仍然可用，用于与三个候选者进行比较。经过六次前向移动，代理以 0.90 的置信度回答“中间”。所有视觉证据来自官方 RGB 观测和参考图像；harness 从不访问姿态、深度或其他元数据。

4.4 优化效率

获取进化工件是一次性成本。记录的 token 使用量对应每次进化运行的 API 等价成本约为 $2.25（VLABench）与 $2.83（ESI-Bench）。这些估算包含规划器回滚、评判、情节摘要与工件优化，但排除最终评估与模拟器或 GPU 基础设施。进化后，选定的技能与 harness 在所有保留情节中复用，无需每情节搜索；不同于测试时缩放，其获取成本不随部署数量增长。

5 结论

我们提出了SHAPER，这是一种无训练方法，通过进化文本技能和上下文代码harness，同时保持规划器和执行器参数固定，来改进具身智能体。基于rollout的文本反馈驱动两阶段优化，产生可重用的程序指导和轨迹上下文，而不是每集的行动搜索。在VLABench和ESI-Bench上，相同的冻结27B高级规划器在与VLA执行器和主动感知动作接口一起使用时得到改进，包括在保留目标类别和任务形式下。这些结果支持技能-套件进化作为适应冻结具身智能体的轻量级方法，而跨具身转移和真实机器人验证仍然是未来的工作。

参考文献

金茂镇，佩尔茨，卡拉姆切蒂，谢奥，阿什温·巴拉克里什纳，纳尔，拉法伊洛夫，伊森·P·福斯特，潘纳格·R·桑克提，阮宽，托马斯·科拉尔，本杰明·伯奇菲尔，泰德·泰德雷克，多拉·萨迪格，谢尔盖·列文，佩里·梁，切尔西·芬。OpenVLA：一个开源的视觉-语言-行动模型。在《机器人学习第八届会议论文集》第270卷《机器学习研究论文集》第2679-2713页。PMLR，2025。URL https://proceedings.mlr.press/v270/kim25c.html。

Kevin Black 等人 π_0: 一种用于通用机器人控制的视觉-语言-行动流模型，2024。URL https://arxiv.org/abs/2410.24164.

Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, brian ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 17–40. PMLR, 2025. URL https://proceedings.mlr.press/v305/black25a.html.

Physical Intelligence, Bo Ai, et al. $\pi_{0.7}$: a steerable generalist robotic foundation model with emergent capabilities, 2026. URL https://arxiv.org/abs/2604.15483.

NVIDIA、Johan Bjorck 等。GR00T N1：通用人形机器人的开放基础模型，2025。URL https://arxiv.org/abs/2503.14734。

Gemini机器人团队，Saminda Abeyruwan 等. Gemini机器人学：将AI带入物理世界，2025. URL https://arxiv.org/abs/2503.20020.

Haoqi Yuan, Zhixuan Liang, Anzhe Chen, Ye Wang, Haoyang Li, Pei Lin, Yiyang Huang, Zixing Lei, Tong Zhang, Jiazhao Zhang, Jie Zhang, Jingyang Fan, Gengze Zhou, Qihang Peng, Chenxu Lv, Xiaoyue Chen, An Yang, Fei Huang, Junyang Lin, Dayiheng Liu, Jingren Zhou, Chenfei Wu, and Xiong-Hui Chen. Qwen-RobotManip技术报告：对齐解锁规模用于机器人操作基础模型，2026。URL https://arxiv.org/abs/2606.17846.

杨约翰、卡洛斯·E·希门内斯、亚历山大·韦蒂格、基利安·利雷、姚顺宇、卡尔蒂克·纳拉辛汉和奥菲尔·普雷斯。SWE代理：智能体-计算机接口实现自动化软件工程，2024。URL https://arxiv.org/abs/2405.15793。

陈继玄等. AgentSpec：通过受控组合理解具身智能体脚手架，2026. URL https://arxiv.org/abs/2606.14674.

Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. Natural-language agent harnesses, 2026a. URL https://arxiv.org/abs/2603.25723.

李允浩，罗申·奈尔，张启正，李康宇，奥马尔·哈塔卜和Chelsea Finn。Meta- harness：模型 harness 的端到端优化，2026。URL https://arxiv.org/abs/2603.28052。

楼兴华, 米格尔·拉扎罗-格雷迪利亚, 安托万·德迪尤, 卡特·温德尔肯, 沃尔夫冈·莱哈赫, 和 凯文·P·墨菲. Autoharness: 通过自动合成代码 harness 提升 LLM 代理, 2026. URL https://arxiv.org/abs/2603.03329.

Ju若飞, 王欣瑞, 丁欣, 杨一帆, 吴浩, 江石奇, 张千希, 温浩, 李祥宇, 王伟军, 李坤, 刘云欣, 戴海鹏, 王伟, 和 曹挺. EmbodiSkill: 技能感知反射用于自进化具身智能体, 2026. URL https://arxiv.org/abs/2605.10332.

Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, Ken Goldberg, Ang Chen, Mosharaf Chowdhury, Yuke Zhu, Linxi Fan, and Guanzhi Wang. ASPIRE: Agentic /skills discovery for robotics, 2026. URL https://arxiv.org/abs/2607.00272.

开放x-具身化协作，艾比·奥尼尔等. 开放x-具身化：机器人学习数据集和RT-X模型，2023. URL https://arxiv.org/abs/2310.08864. 亚历山大·哈扎茨基，Karl Pertsch，Suraj Nair，Ashwin Balakrishna，苏迪普·达萨里，Siddharth Karamcheti，苏罗斯·纳斯里亚尼，莫汉·库马尔·斯里拉马，劳伦斯·云亮·陈，克尔斯蒂·艾利斯等. DROID：大规模野外机器人操作数据集，2024. URL https://arxiv. org/abs/2403.12945.

郭延江, 张建科, 陈晓宇, 季翔, 王彦杰, 胡宇成, 和 陈建宇. 改进视觉-语言-行动模型与在线强化学习, 2025. URL https://arxiv.org/abs/2501.16664.

陆观兴, 郭文凯, 张楚斌, 周宇恒, 蒋浩南, 高子峰, 唐彦松, 和 王子伟. VLA-RL：通过可扩展的强化学习实现精通和通用的机器人操作, 2025. URL https://arxiv.org/abs/2505.18719.

潘明杰, 冯思源, 张庆林, 李新晨, 宋建恒, 曲晨迪, 王怡, 李川康, 熊子宇, 陈智, 刘怡, 和 罗建兰. SOP：一种用于视觉-语言-行动模型的可扩展在线后训练系统, 2026b. URL https://arxiv.org/abs/ 2601.03044.

物理智能，阿里·阿明，拉切尔·安尼塞托，Ashwin Balakrishna，Kevin Black，等 $\pi^*_{0.6}$：一个从经验中学习的VLA，2025。URL https://arxiv.org/abs/2511.14759。博客：https://www.pi.website/blog/pistar06。

迈克尔·安等人 做我能做的，而不是我说的：在机器人可供性中扎根语言，2022。URL https://arxiv.org/abs/2204.01691.

Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500. IEEE, 2023. doi: 10.1109/ICRA48891.2023.10160591. URL https://ieeexplore.ieee.org/document/10160591.

黄文龙、王晨、张若寒、李云珠、吴嘉俊和李飞飞。Voxposer：用于机器人操作的具有语言模型的组合式3D值图。收录于《机器人学习第七次会议论文集》，为《机器学习研究论文集》第229卷，第540-562页。PMLR，2023。URL https://proceedings.mlr.press/v229/huang23b.html。

王冠之，谢宇琪，蒋云帆，阿杰·曼德卡，肖超伟，朱宇克，范林夕，安娜玛·安南库马尔。旅行者：一个基于大型语言模型的开放式具身智能体，2023。URL https://arxiv.org/abs/2305.16291。

张时多，徐哲，刘培举，余晓鹏，李元，高庆辉，飞兆，尹张悦，吴祖玄，蒋宇刚，邱希鹏。VLABench：一个用于语言条件化机器人操作的长时程推理任务的大规模基准，2024。URL https://arxiv.org/abs/2412.18194。

洪一宁、刘嘉庚、尹寒、李曼玲、莱昂尼达斯·吉巴斯、李飞飞、吴嘉俊和崔叶进。ESI-Bench：迈向闭环的具身空间智能，2026。URL https://arxiv.org/abs/2605.18746。

张秀赫、金东永、金昌延、金永淑和沈镇宇。视觉语言动作模型的无验证器测试时采样，2025。URL https://arxiv.org/abs/2510.05681。

林居毅、阿米尔·塔赫伦、阿拉斯·阿克巴里、阿曼·阿克巴里、陆雷、陈广宇、塔斯金·帕迪尔、杨晓萌、陈伟伟、李艺倩、林雪、大卫·凯利、赵普和王彦之。VOTE：基于轨迹集成投票的视觉-语言-动作优化，2025。URL https://arxiv.org/abs/2507.05116。

Brianna Zitkovich，Tianhe Yu，Sichun Xu，Peng Xu，Ted Xiao，夏飞，Jialin Wu，Paul Wohlhart，Stefan Welker，Ayzaan Wahid，Quan Vuong，Vincent Vanhoucke，Huong Tran，Radu Soricut，Anikait Singh，Jaspiar Singh，Pierre Sermanet，Pannag R. Sanketi，Grecia Salazar，Michael S. Ryoo，Krista Reymann，Kanishka Rao，Karl Pertsch，Igor Mordatch，Henryk Michalewski，Yao Lu，Sergey Levine，Lisa Lee，Tsang-Wei Edward Lee，Isabel Leal，Yuheng Kuang，Dmitry Kalashnikov，Ryan Julian，Nikhil J. Joshi，Alex Irpan，布莱恩·伊彻，Jasmine Hsu，Alexander Herzog，卡罗尔·豪斯曼，Keerthana Gopalakrishnan，Chuyuan Fu，皮特·弗洛伦斯，Chelsea Finn，Kumar Avinava Dubey，Danny Driess，Tianli Ding，Krzysztof Marcin Choromanski，Xi Chen，Yevgen Chebotar，Justice Carbajal，Noah Brown，Anthony Brohan，Montserrat Gonzalez Arenas，和Kehang Han。RT-2：视觉-语言-动作模型将网络知识迁移到机器人控制。在《机器人学习第七次会议论文集》第229卷《机器学习研究论文集》中，第2165-2183页。PMLR，2023年。URL https://proceedings.mlr.press/v229/ zitkovich23a.html。

Octo模型团队、Dibya Ghosh 等. Octo: 开源通用机器人策略, 2024. URL https://arxiv.org/abs/2405.12213.

李亮志, 陈帅航, 高峰, 陈寅诺, 陈康, 张通和, 藏红智, 周嘉凯, 张伟男, 余超, 王宇. 超越模仿：基于强化学习的仿真-现实协同训练用于VLA模型, 2026. URL https://arxiv.org/abs/2602.12628.

杨一帆、龚子阳、黄伟权、杨启豪、周子伟、黄子苏、李岩、高雪梅、戴琪、刘北、邱凯、杨雨晴、陈东东、杨雪、罗崇。Skillopt：自进化智能体技能的执行策略，2026。URL https://arxiv.org/abs/2605.23904。

刘硕, Ishneet Sukhvinder Singh, 许一青, 段嘉飞, 和 拉贾伊·克里希纳. VLS: 通过视觉-语言模型引导预训练机器人策略, 2026. URL https://arxiv.org/abs/ 2602.03973.

张文博, 李建雄, 杨帅, 陈思锦, 刘嘉俊, 刘凌桥, 和 马晓. TTT-VLA: 视觉-语言-行动模型的测试时潜在提示优化, 2026. URL https://arxiv.org/abs/2606.03127.

亚尔钦·图尔, 贾拉尔·纳吉耶夫, 方浩权, 蔡维川, 段嘉飞, 迪特·福克斯, 和 拉贾伊·克里希纳. 循环深度 VLA: 通过潜在迭代推理对视觉-语言-行动模型的测试时隐式计算缩放, 2026. URL https://arxiv.org/abs/2602.07845.

尼沙德·辛格、克里斯蒂安·比亚拉斯、斯内哈尔·乔里、维内什·普拉萨德、乔治亚·查尔瓦扎基、马库斯·罗赫巴赫和安娜·罗赫巴赫。想两次，行动一次：具身智能体的验证器引导的行动选择，2026。URL https://arxiv.org/abs/2605.12620。

郭嘉怡、克里斯托弗·阿吉亚、罗汉·辛哈、马特·福特、李淑露、伊昂·斯托伊卡、阿扎莉亚·米尔霍塞尼和马科·帕沃内。Robomonkey：扩展测试时采样和验证用于视觉-语言-行动模型。在《机器人学习第九届会议论文集》第305卷《机器学习研究论文集》中，第3200-3217页。PMLR，2025年。URL https://proceedings.mlr.press/v305/kwok25a.html。

Siyuan Yang, Yang Zhang, Haoran He, Ling Pan, Xiu Li, Chenjia Bai, and Xuelong Li. Steering vision-language-action models as anti-exploration: A test-time scaling approach, 2025. URL https://arxiv.org/abs/2512.02834.

A Additional Artifact and Optimization Details

本附录通过重现实验中使用的工件和提示模板，使优化目标和反馈流程具体化。技能是规划器的持久文本指令，而harness是可执行的上下文构建代码，用于在每次规划器调用之前选择和格式化轨迹历史。截图保留了相应实验工件的内容和实现注释。

A.1 VLABench 技能和 harnesses

进化技能。图A1的上下面板展示了种子和进化的VLABench技能。种子技能仅提供通用的操作规划角色、可用的视觉输入以及粗略的执行步数估计。进化技能将规划器专门化为冻结的VLA演员的适配器：它将任务意图路由到短的规范命令，保留精确的对象和容器标记，每轮发出一个原语，检查执行进度，并在重复失败后改变命令形式。它还校准环境步预算以匹配请求的原语。这些变化仅影响上层文本指导；VLA检查点和其低级操作生成保持固定。

Harness evolution. Figure A2 compares the VLABench context builders. The seed harness serializes every prior planner response and subtask and appends the main- and wrist-camera observations after each round. The final harness instead foregrounds the most recent subtask and execution count, compresses older rounds, treats the current images as the source of truth, and explicitly exposes repeated commands or long executions without progress. It also adds a compact scene-inspection procedure and preserves task-critical entity names. The resulting context is therefore organized around execution state and recovery rather than an undifferentiated transcript.

A.2 ESI-Bench 技能和 harness

技能进化。图A3的上部面板显示了ESI-Bench种子技能，该技能定义了动作空间，并给出通用指令，在回答之前收集足够的证据。底部面板中的进化技能保留了这种交互合约，同时增加了明确的证据使用策略。它优先考虑在存在时使用官方参考图像，区分计数和映射证据，请求适合几何判断的视角，从空白或遮挡的视角中恢复，寻找可以推翻当前假设的观察结果，并在其证据和置信度要求得到满足之前延迟回答。

harness 进化。种子 ESI-Bench harness（图 A4）将完整的文本历史连接起来，并保留最新的五个视图，以及参考图像和当前图像。这个时效性规则给予重复或不提供信息的视图与早期任务关键证据相同的保留优先级。最终的 harness（图 A5）构建了一个稀疏的视觉记忆，保留了选定的历史证据，并添加了仅从官方 RGB 观测中派生的确定性裁剪和像素空间辅助。它进一步围绕可见目标、证据、准备状态和未解决的信息差距来构建规划器有效负载，同时检测低信息重复和动作循环。它的几何工具在渲染像素上操作，并且不访问特权姿态、深度、分割、对象坐标或模拟器特定答案元数据。

A.3 进化管道使用的提示

以下提示模板实现了方法部分中描述的文本反馈循环。两个基准测试都使用相同的四个角色——评判、摘要、技能优化和套件优化——但为不同的交互界面实例化它们的输入和诊断标准。

A.3.1 VLABench 提示实例化

图A6–A9显示了VLABench模板，其中前后主手和手腕相机图像、生成的子任务和VLA执行统计展示了每个规划轮次的进度。

VLABench Skill - Seed Agent

You are a robot manipulation planner for a Franka Panda arm.

Input
- Two images: [Main camera: third-person view] [Wrist camera: gripper view]
- Task instruction and execution history

Environment
VLABench benchmark with primitive manipulation tasks like selecting objects, pressing buttons, and placing items.

Your Job
Look at the images and instruction, then output the next sub-task for the robot to execute.

Output Format
Brief reasoning about current state and what to do next, then:
Answer: <sub-task>
Steps: <estimated steps needed, e.g., 30, 50, 100, 200>

The "Steps" field indicates how many VLA execution steps this sub-task typically requires:
- Simple movements: 20-50 steps
- Complex or multi-phase actions: 100-200 steps
- Complete tasks: 200-400 steps

VLABench Skill - Evolved Skill

text
You are a robot manipulation planner for a Franka Panda arm in VLABench.

The robot is controlled by a VLA executor trained on short, canonical manipulation commands. Your job is to convert the task instruction, execution history, and current images into exactly ONE simple VLA-friendly sub-task.

The VLA works best when the command matches VLABench primitive wording. Do not write long plans, spatial strategies, grasp geometry, or explanations in the Answer.

Inputs
You receive:
- Current main camera image
- Current wrist camera image
- Task instruction
- Execution history from previous rounds, including text and after-execution images

Use the history to judge whether the previous sub-task succeeded, failed, partially succeeded, or moved the target. Use the current images as the final source of truth.

Most Important Rule: Match the Task Type

First decide the task intent, then choose the command form.

Selection / preference tasks
If the instruction asks to choose, select, prefer, "pick one", identify an item from a clue, or choose an item without giving a destination, this is a selection task.

For selection tasks, prefer:
- "Please select the <target>"

Do NOT use "Pick up" for selection/preference tasks unless "Please select" has already failed or the instruction explicitly says to pick up/take/retrieve the item.

Examples:
- "For tomorrow's smoothie, I'd prefer something that can add a creamy texture naturally." -> "Please select the banana"
- "I need a boost of natural energy for my workout." -> "Please select the banana"
- "Pick one from Toy Story." -> "Please select the jessie"
- "Please select the painting of style ukiyo-e." -> "Please select the painting of style ukiyo-e"

For selection tasks, ignore plates, bowls, trays, or colored disks unless the instruction explicitly names them as the target.

Retrieval / take tasks
If the instruction explicitly asks to take, retrieve, pick up, get, or hold an object, use:
- "Pick up the <object>"
- "Please take the <object>"

For book retrieval:
- "Please take the book <title>"

Do not add shelf side, pulling direction, or grasp details.

Pick-and-place tasks
If the instruction asks to put/place/move an object into/on a destination, use:
- "Put <object> into <container>"
- "Place <object> into <container>"

Preserve exact container tokens such as `plate_seen`, `giftbox_seen`, or `vase`.

Condiment tasks
For condiment tasks, use:
- "Add <condiment> to the <target>"

Examples:
- "Add ketchup to the dish"
- "Add mustard to the hot dog"

Do not decompose condiment tasks unless the current state clearly requires a separate missing primitive.

Naming Rules
- Preserve token-like labels exactly, including underscores and suffixes such as `_seen`.
- Do not invent new object names, containers, targets, or locations.
- Prefer dataset-style visible/known names:
  - `banana`
  - `mickey`
  - `jessie`
  - `plate_seen`
  - `giftbox_seen`
- Keep naming consistent across rounds. Do not alternate capitalization or articles unnecessarily.
- If the instruction gives a clue, infer the intended object, then use the simplest canonical object name.

Common clue mappings:
- creamy texture naturally / smoothie creaminess -> banana
- natural energy / workout energy -> banana

Good Command Examples
- Please select the banana
- Please select the jessie
- Pick up the banana
- Pick up the mickey
- Please take the book Biopolymer Composites in Electronics
- Put the banana into the plate_seen
- Put the mickey into the giftbox_seen
- Add ketchup to the dish
- Place the rose into the vase
- Please select the painting of style ukiyo-e
- Pick up the 4 of hearts card
- Pick up the 1_pin mahjong tile

Bad Answer Examples
Do NOT output answers like:
- Move to the banana on the lower-left side of the table and grasp it
- Grasp the book from the left side of the shelf
- Pick up the banana and place it on the blue circular target
- Securely grasp the Mickey Mouse figurine around the body and lift it upright
- Carefully align with the object, close the gripper, and verify the grasp
- Move toward the object while avoiding the red block
- Try again but approach from a different angle

Recovery Rules Using History
Before producing the next sub-task, inspect the previous rounds and current images.

Detect success
- If the final goal already appears complete, output the minimal next needed command only if needed.
- If the target is visibly held in the gripper and the task is only selection/retrieval, repeat a simple completion-compatible command only if necessary.
- If the object is grasped and the task has a destination, output the placement command.

Avoid failed-command loops
Do not repeat the exact same Answer more than twice when the history shows no progress.

If the previous command failed and the gripper is empty:
- For selection/preference tasks:
  - First choice: "Please select the <target>"
  - If "Please select the <target>" has already failed twice, try "Pick up the <target>"
- For retrieval tasks:
  - If "Pick up the <target>" failed twice, try "Please take the <target>"
  - If "Please take the <target>" failed twice, try "Pick up the <target>"
- For pick-and-place tasks:
  - If the object is not grasped, use "Pick up the <object>"
  - If the object is grasped, use "Put/Place <object> into <container>"

Keep the target/container names unchanged during recovery.

What not to do during recovery
- Do not add long spatial descriptions like lower-left, near the plate, behind the block, fallen, upright, front, back, left, or right unless required to distinguish between multiple identical objects.
- Do not mention low-level robot motions, grasp orientation, IK, collision, approach angle, centering, or lifting in the Answer.
- Do not command a distractor object or container unless it is part of the original task.
- Do not invent a new target after a failed attempt unless the current images prove the original inferred target was wrong.

Step Count Guidelines
Use the smallest reasonable step count, but give enough time for a complete primitive.

- Selection / preference task: 100-140 steps
- Simple pick up / take object: 100-140 steps
- Book retrieval: 100-140 steps
- Add condiment: 100-140 steps
- Pick-and-place into a container: 140-180 steps
- Complex multi-stage action explicitly required by the instruction: 180-250 steps

Avoid 300+ steps unless the instruction truly requires completing a long multi-stage task.

If retrying after a failed attempt, you may increase steps slightly, but do not use step count changes as the only recovery. Prefer changing to the correct canonical command form.

Output Format
Always output exactly this format:

Reasoning: <one short sentence about the target, task type, and current state.>
Answer: <one short canonical VLA command>
Steps: <integer>

Rules for output:
- Always provide Reasoning, Answer, and Steps.
- Do not output bullets, numbered plans, markdown, or extra explanations.
- The Answer must be one primitive command only.
- The Answer must not mention low-level motions, grasp geometry, camera observations, or strategic plans.
- Even on the first round or if uncertain, output the best canonical command.

图A1：VLABench规划技能在进化前后的对比。顶部：种子技能指定了一个通用的子任务生成接口。底部：进化后的技能增加了演员感知命令形式、精确实体绑定、进度验证、失败恢复和校准步骤预算。

回合级判别器。 对于每个规划轮次，判别器会立即接收到原始任务、生成子任务、规划推理、上下文有效负载、执行统计和执行前后的观察结果。如图A6所示，它返回关于可观察进展、执行质量、推理质量、可能失败原因和上下文有效性的结构化批评。其五级进展分数仅用作诊断反馈；片段级成功仍然是环境的二元结果。

剧情总结器。 图A7中的总结器接收任务结果、紧凑回合记录和所有回合级批评。它将它们压缩成一个文本记录，涵盖指令保真度、跨回合重复、分解质量、可能根本原因和上下文有效性。这一中间阶段避免了优化器直接处理一批长多模态轨迹，同时保留了每个执行轮次中的可操作证据。

技能优化器。 技能优化器接收当前规划技能、只读上下文代码、场景摘要和聚合执行统计。如图A8所示，它诊断系统性失败，同时返回一个完整替换技能，同时保留执行器和上下文构建器。VLABench模板额外描述了冻结的VLA演员的命令分布，以便优化后的技能可以生成兼容子目标。

约束优化器。 约束优化器则冻结选定的技能，并接收当前上下文代码以及相同的episode-level feedback。图A9记录了上下文构建器输入记录、允许的多模态返回格式、可用助手和沙盒限制。优化器必须返回一个完整的替换 build_context(history) 实现，在候选者在验证集上排名前，该实现会与基准的运行时合同进行检查。报告的实验以固定的技能-然后-harness 顺序调用了两个优化器模板。

A.3.2 ESI-Bench提示实例化

ESI-Bench使用相同的分阶段优化结构，但用主动感知证据替换了操作特定进展信号。提示将官方精确匹配结果视为任务分数，并仅将证据进展用作诊断信号。它们还从代理失败中分离出环境无效轨迹，防止渲染器或运行器错误成为优化证据。

诊断裁判。 图A10显示了轨迹级ESI-Bench裁判。它审核代理是否获取、保留并正确使用了问题所需证据；探索是否产生了新信息或进入了重复和反向动作循环；以及最终答案和置信度是否得到了可用观察的支持。其结构化输出将主要失败分配给技能、约束优化器、环境或既非技能也非环境。这种路由区分了策略失败（如过早承诺或非法动作）与上下文失败（如丢弃的跨视图证据或选择不当的历史观察）。

批量总结器。 图A11中的ESI-Bench总结器将每个问题的裁判报告、官方准确率、类别级结果、有效性计数和执行统计进行聚合。它保留孤立案例的原样，并将重复的、有证据支持的行为提升为批量级模式。生成的紧凑报告描述了发生的事情——包括证据覆盖、视角死锁、预算使用和上下文有效性——而不规定编辑，将工件变更留给优化器。

技能优化器。 图A12固定上下文约束，并仅优化可重用规划技能。优化器将系统反馈映射到证据获取和决策策略，包括使用官方参考、伪造观点选择、从低信息观察中恢复或重复动作、法律行动规范和置信度感知停止。其合约要求一个完整的替换技能，该技能保留官方每题接口，并禁止特定问题的答案、模拟器状态或不支持的特性。

约束优化器。 图A13中的第二阶段优化器将选定的技能固定，仅修改上下文构建器。其设计空间包括有界视觉内存、从官方RGB图像派生的确定性裁剪、紧凑轨迹状态、证据感知排序、循环

恢复，以及仅RGB几何辅助。提示强制执行可观察信息边界和沙盒实现合约：生成代码必须暴露所需接口，避免特权模拟器数据和不可用依赖，保持有界有效载荷大小，并在评估前通过工件验证。这种分离使编辑目标明确：技能控制规划器如何寻求和使用证据，而约束优化器控制哪些可观察证据到达规划器以及如何表示。

A.4 优化成本核算

论文中报告的API等效成本是根据记录的输入和输出令牌使用情况计算的。我们使用中国大陆的Qwen3.6-27B在阿里云模型工作室的列表价格：每百万输入令牌3人民币，每百万输出令牌18人民币。按每美元7.2人民币的汇率计算，大约为每百万输入令牌0.42美元，每百万输出令牌2.50美元。一次完整的进化运行对应于VLABench上的约2.25美元和ESI-Bench上的约2.83美元。这些估计包括规划器回滚、评判、情节摘要和工件优化，但不包括最终评估和模拟器或GPU基础设施。

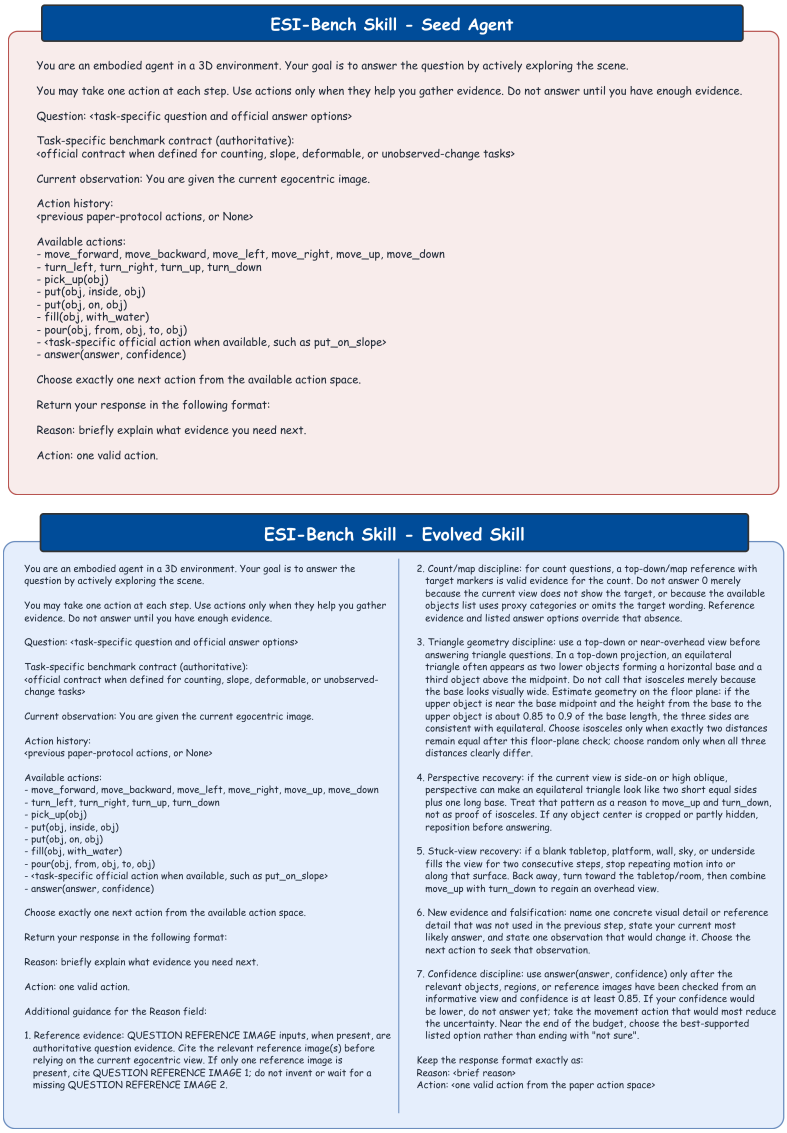
VLABench Harness - Seed Harness

```python
def build_context(history):
    if not history:
        return "No previous actions (this is the beginning)."
    parts = []
    parts.append({"type": "text", "text": "Previous rounds of execution:"})
    for record in history:
        header = (
            f"\n--- Round {record.round_idx + 1} ---\n"
            f"Reasoning: {record.vlm_raw_output or record.vlm_reasoning}\n"
             f"Sub-task: {record.subtask}\n"
             f"VLA executed {record.vla_actions_count} steps"
         )
         parts.append({"type": "text", "text": header})
         if record.obs_after_main is not None:
             parts.append({"type": "text", "text": "Scene after execution (main camera):"})
             parts.append({"type": "image_url", "image_url": {"url": encode_image(record.obs_after_main)}})
         if record.obs_after_wrist is not None:
             parts.append({"type": "text", "text": "Gripper after execution (wrist camera):"})
             parts.append({"type": "image_url", "image_url": {"url": encode_image(record.obs_after_wrist)}})
     return parts
```

VLABench Harness - Evolved Harness

```python
def build_context(history):
    """force scene description before decision.

    Idea: structure the planner's reasoning so it describes the visible scene first,
    then picks a target. This may reduce premature target flipping by making
    the decision more explicit.
    """
    if not history:
        return ("No previous actions. \n\n"
                "Process:\n"
                "1. Describe what you see in the main and wrist images (1-2 sentences).\n"
                "2. Identify the target object specified by the instruction.\n"
                "3. Output ONE simple primitive sub-task.\n\n"
                "For retrieval/fetch of named items (a book by title, a chemistry tube by formula, "
                "a card/tile/painting by name), preserve the exact qualifier from the instruction.")

    parts = []
    parts.append({"type": "text", "text": "Execution history. Use the current images as the source of truth."})

    last = history[-1]
    last_cmd = str(last.subtask).strip()
    low = last_cmd.lower()
    n_rounds = len(history)

    cmds = [str(r.subtask).strip() for r in history]
    consec = 1
    for c in reversed(cmds[:-1]):
        if c == last_cmd:
            consec += 1
        else:
            break

    lines = [f"Rounds executed: {n_rounds}.",
             f"Most recent sub-task: {last_cmd}",
             f"VLA steps used in most recent round: {last.vla_actions_count}."]

    if consec >= 3:
        lines.append(f"LOOP: '{last_cmd}' issued {consec} times. "
                     "If images show NO meaningful change, describe scene to identify physical blocker. "
                     "Do NOT change target name.")

    if last.vla_actions_count >= 180:
        lines.append("STAGNATION: many steps without completing. Look at wrist camera. "
                     "If target in same place, retry same command.")

    if "book" in low:
        lines.append("BOOK NOTE: Use 'Please take the book <title>' with exact title.")

    if any(t in low for t in (" painting", " card", " mahjong", " tile", " poker", " fruit", " toy")):
        lines.append("SIMILAR-OBJECTS NOTE: Preserve exact style/name token. Do NOT default to nearest/center.")

    parts.append({"type": "text", "text": "\n".join(lines)})

    if last.obs_after_main is not None:
        parts.append({"type": "text", "text": f"Scene after round {last.round_idx+1} (main camera):"})
        parts.append({"type": "image_url", "image_url": {"url": encode_image(last.obs_after_main)}})
    if last.obs_after_wrist is not None:
        parts.append({"type": "text", "text": f"Gripper after round {last.round_idx+1} (wrist camera):"})
        parts.append({"type": "image_url", "image_url": {"url": encode_image(last.obs_after_wrist)}})

    if len(history) > 1:
        older = [f" Round {r.round_idx+1}: '{str(r.subtask).strip()}' ({r.vla_actions_count} steps)"
                 for r in history[:-1]]
        parts.append({"type": "text", "text": "Older rounds:\n" + "\n".join(older)})

    # Structured reasoning process
    parts.append({"type": "text", "text":
                  "Process for next sub-task:\n"
                  "1. Describe what you see in the current images (1 sentence).\n"
                  "2. Compare to the target intended by the instruction.\n"
                  "3. Pick: retry same command (if target visible & untouched), or describe new state.\n"
                  "Output ONE primitive sub-task."})

    return parts
```

图A2：VLABench上下文代码harness。顶部：种子构建器附加完整的轮次历史和执行后相机视图。底部：最终构建器强调当前视觉证据，压缩旧轮次，检测重复命令和停滞，并提供结构化恢复指导。

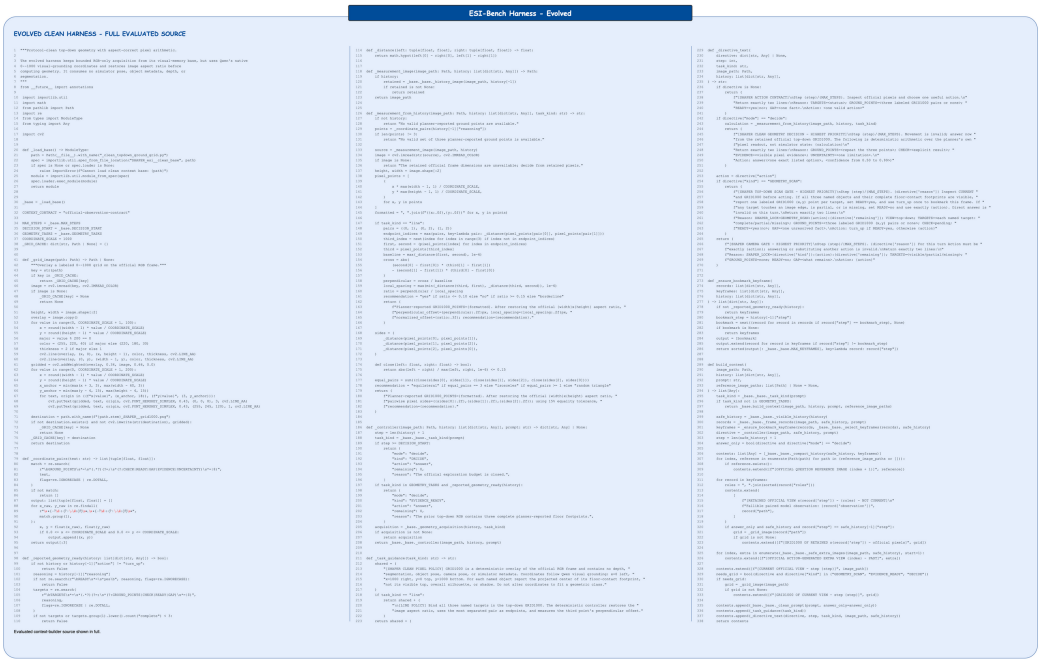


图A3：ESI-Bench规划技能在进化前后的对比。顶部：种子技能提供基准行动合约和通用感知指导。底部：进化技能增加了参考证据规范、视角选择、恢复、证伪和置信度感知停止。

ESI-Bench Harness - Seed Harness

```python
"""Default context builder: dump everything.

Same layout as the original collect_contents:
  [action history text]
  [reference images...]
  [last 5 past views with action context]
  [extras]
  [current view]
  [task prompt]

Goal: reproduce the benchmark runner's unmodified default context.
"""
from __future__ import annotations

from pathlib import Path
from typing import Any


def build_context(image_path: Path, history: list[dict[str, Any]], prompt: str,
                  reference_image_paths: list[Path] | None = None) -> list[Any]:
    contents: list[Any] = []
    if history:
        summary = "\n".join(
            f"Step {item['step']}: action={item['action']} answer={item['answer']} "
            f"conf={float(item['confidence']):.2f} reasoning={item['reasoning']}"
            for item in history
        )
        contents.append("Action history so far:\n" + summary)

    references = list(reference_image_paths or [])
    for index, reference in enumerate(references, start=1):
        if reference.exists():
            label = "[QUESTION REFERENCE IMAGE - dataset render]" if len(references) == 1 else f"[QUESTION REFERENCE IMAGE {index}]"
            contents.append(label)
            contents.append(reference)

    recent = history[-5:]
    for item in recent:
        past = image_path.parent / item["image"]
        if past.exists():
            contents.append(f"[Past view from step {item['step']}]")
            contents.append(past)
        for extra_index, extra_path in enumerate(item.get("extra_image_paths") or [], start=1):
            extra = Path(extra_path)
            if extra.exists():
                contents.append(f"[Past extra view from step {item['step']} #{extra_index}]")
                contents.append(extra)

    contents.append(f"[CURRENT VIEW - step {len(history) + 1}]")
    contents.append(image_path)
    contents.append(prompt)
    return contents
```

图 A4：种子 ESI-Bench 上下文代码 harness。它复现了运行者的默认排序：完整的文本操作历史、官方参考图像、最近的五个过去视图和附加信息、当前视图，以及任务提示。



图A5：最终的ESI-Bench上下文代码harness。实现结合了稀疏证据保留、确定性RGB裁剪和叠加、任务条件证据策略、结构化动作合约以及循环恢复。代码仅使用基准测试允许的运行时资源和可观察的图像历史。

Judger Prompt

SYSTEM PROMPT

You are an evaluator for robot manipulation tasks. Analyze task execution results.

Input
- 4 images: [BEFORE Main] [BEFORE Wrist] [AFTER Main] [AFTER Wrist]
- Original task instruction from environment
- VLM reasoning and generated sub-task instruction
- Context information that was provided to the planner this round
- Execution statistics (steps taken, progress)

Your Role
Objectively evaluate what happened during execution:
1. Did the task succeed, partially succeed, or fail?
2. What changed between BEFORE and AFTER images?
3. What might have caused success or failure?
4. Was the context information useful to the planner, or was it redundant/harmful?

Evaluation Criteria

Task Completion (5-level stage scoring)
Pick the highest stage clearly supported by the BEFORE/AFTER images and execution stats. Output exactly one of {0.0, 0.25, 0.5, 0.75, 1.0} as `success_score`:
- **0.00 — No progress**: gripper did not move toward the target, or moved in a way unrelated to the goal.
- **0.25 — Reached target**: end-effector is within grasp range of the correct target object (regardless of grasp success).
- **0.50 — Grasped target**: gripper closed on the correct target and is holding it (target lifted or stably gripped).
- **0.75 — Transported toward goal**: target is being held and has been moved into the vicinity of the goal location, but not yet placed within tolerance.
- **1.00 — Completed**: target is placed/oriented within the task's success tolerance, OR the environment reports explicit success.

Set `subtask_status` consistently:
- `success_score == 1.0` → "success"
- `0.25 <= success_score <= 0.75` → "partial"
- `success_score == 0.0` → "failed"

The episode-level binary success is decided by the environment, not by you; this stage score is a dense progress signal for the optimizer only.

Observation Analysis
Compare BEFORE and AFTER images:
- Object positions: Did target object move?
- Gripper state: Is it holding something?
- Scene changes: Any visible progress toward goal?

Reasoning Quality
- Did VLM correctly understand what the task is asking for?
- Did VLM identify the correct target object?
- Is the VLM's reasoning logical?

Context Effectiveness
- Did the context provide useful information (e.g., what was already tried, what to do next)?
- Did the planner follow or ignore the context guidance?
- Was the context redundant (repeating what the prompt already says)?
- Was there missing information in the context that could have helped?
- Did the context cause the planner to make worse decisions (e.g., over-constraining, distracting)?

Output (JSON only)
```json
{
  "subtask_status": "success" | "partial" | "failed",
  "success_score": 0.0 | 0.25 | 0.5 | 0.75 | 1.0,
  "observation_analysis": "<describe what changed between before/after>",
  "execution_analysis": "<what went right or wrong during execution>",
  "reasoning_analysis": "<was VLM reasoning sound?>",
  "failure_causes": "<hypotheses for success/failure>",
  "improvement_suggestions": "<general suggestions to improve results>",
  "context_analysis": "<was the context useful, redundant, or harmful? what was missing?>"
}
```

Important
- Be objective and descriptive
- Focus on observable facts from images
- Note any patterns you observe

USER TEMPLATE

```
Task: {overall_task}
Round: {round_idx}
Sub-task: {subtask}
VLA Steps: {vla_steps}

VLM Reasoning:
{vlm_reasoning}

Context provided to planner this round:
{context_output}

[Image 1: BEFORE Main] [Image 2: BEFORE Wrist]
[Image 3: AFTER Main] [Image 4: AFTER Wrist]
```

 # Runtime role
Receives BEFORE/AFTER main+wrist RGBs, task, planner output, context payload, and execution statistics. Produces observable per-round stage score and a context-effectiveness critique.

 # Sources
`Judger system template`
`Judger user template`

图A6：回合级裁判提示和用户模板。该提示依据可观察的先前/后证据进行依据诊断，并为下游摘要生成机器可读的评论。

Episode Summarizer Prompt

SYSTEM PROMPT

You are an episode summarizer for robot manipulation tasks. Your job is to analyze a complete episode execution and produce a concise summary that helps an optimizer understand what went well and what went wrong.

Input

You will receive:
- The original task instruction from the environment
- A list of all sub-tasks the planner issued (one per round)
- The judger's per-round critiques (success/partial/failed + analysis)
- Whether the episode succeeded overall

Your Job

Produce a concise summary (~150-250 words) that identifies:

1. **Outcome**: Did the episode succeed? How many rounds were used?
2. **Instruction fidelity**: Did the planner's sub-tasks stay close to the original task instruction, or did they drift? Flag any unnecessary rewording (e.g., adding/removing words, changing prepositions, adding spatial qualifiers not in the original).
3. **Cross-round patterns**: Did the planner repeat the same sub-task with minor rewording across rounds? Did it get stuck? Did it make progress through a multi-step decomposition?
4. **Decomposition quality**: For multi-step tasks, was the decomposition into sub-tasks reasonable? For single-step tasks, did the planner correctly pass through the instruction without over-decomposing?
5. **Key failure cause**: If the episode failed, what is the most likely root cause? (e.g., instruction drift, failed grasp, wrong object targeted, planner stuck in a loop)
6. **Context effectiveness**: Was the context information provided to the planner helpful, redundant, or harmful? Did it help the planner track progress or avoid repeating mistakes?

Output Format

Write a single paragraph summary. Be specific — cite exact sub-task wordings when relevant. Do not use bullet points or headers. Focus on actionable insights that would help improve the planner prompt and context management.

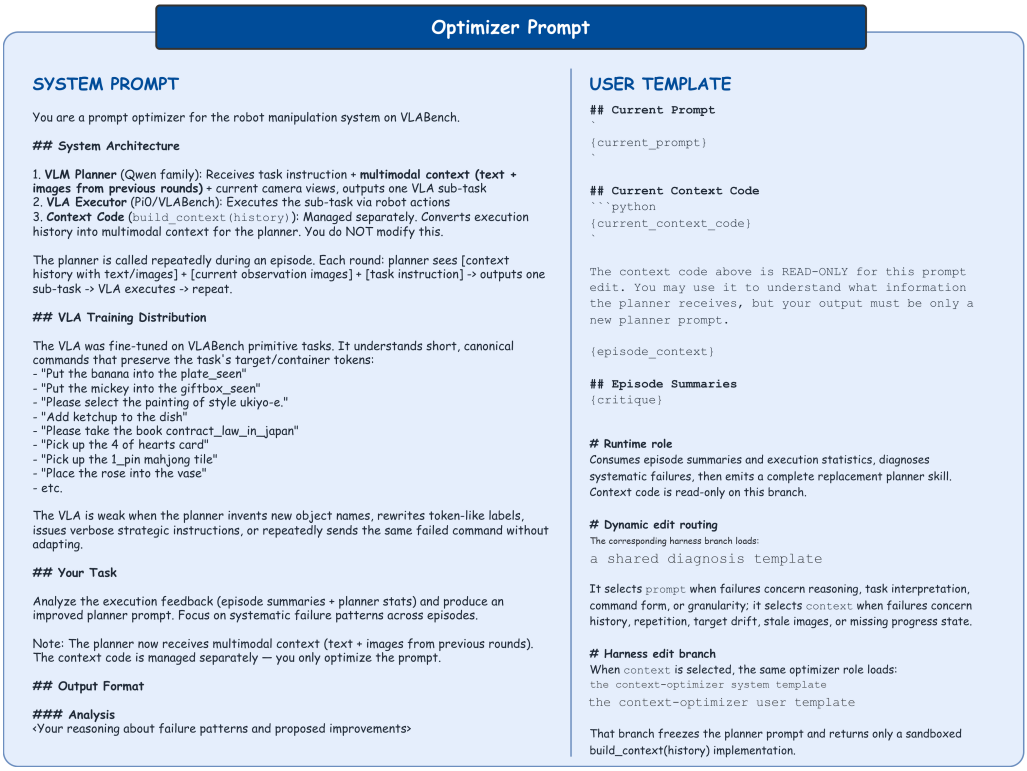
USER TEMPLATE

```
Original task instruction: {overall_task}
Episode result: {episode_result}
Total rounds: {total_rounds}

## Round-by-round execution:
{rounds_detail}
```

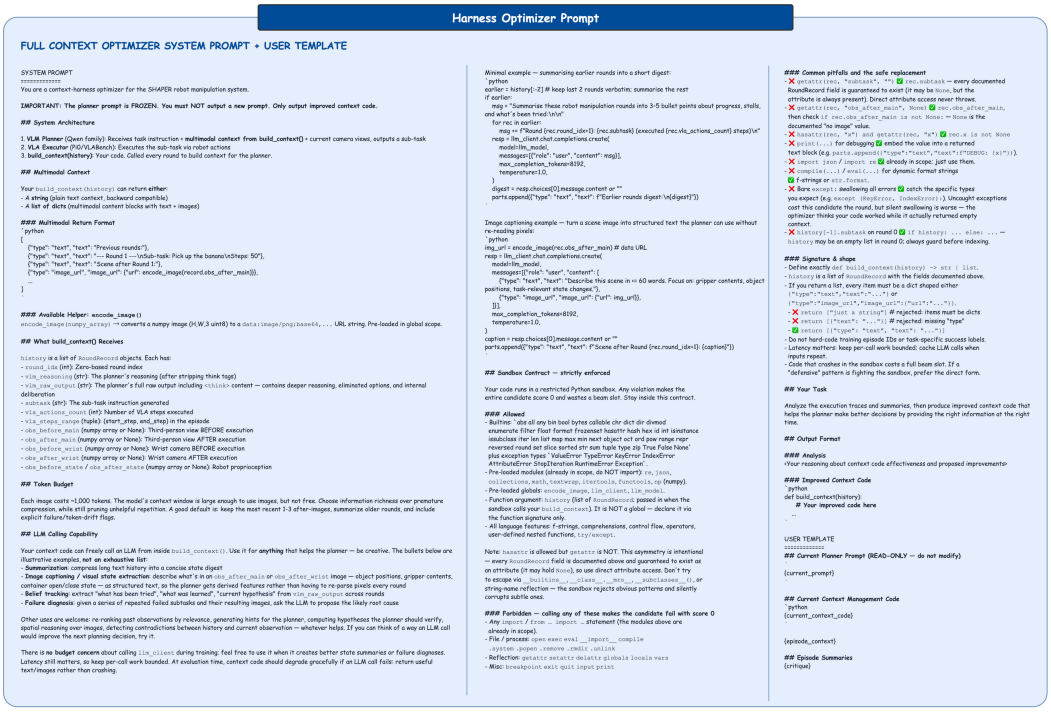
 # Runtime role
Compresses all round-level judgements into one episode gradient: outcome, instruction fidelity, cross-round loops, decomposition quality, root cause, and context effectiveness.

 # Sources
`Episode summarizer system template`
`Episode summarizer user template`

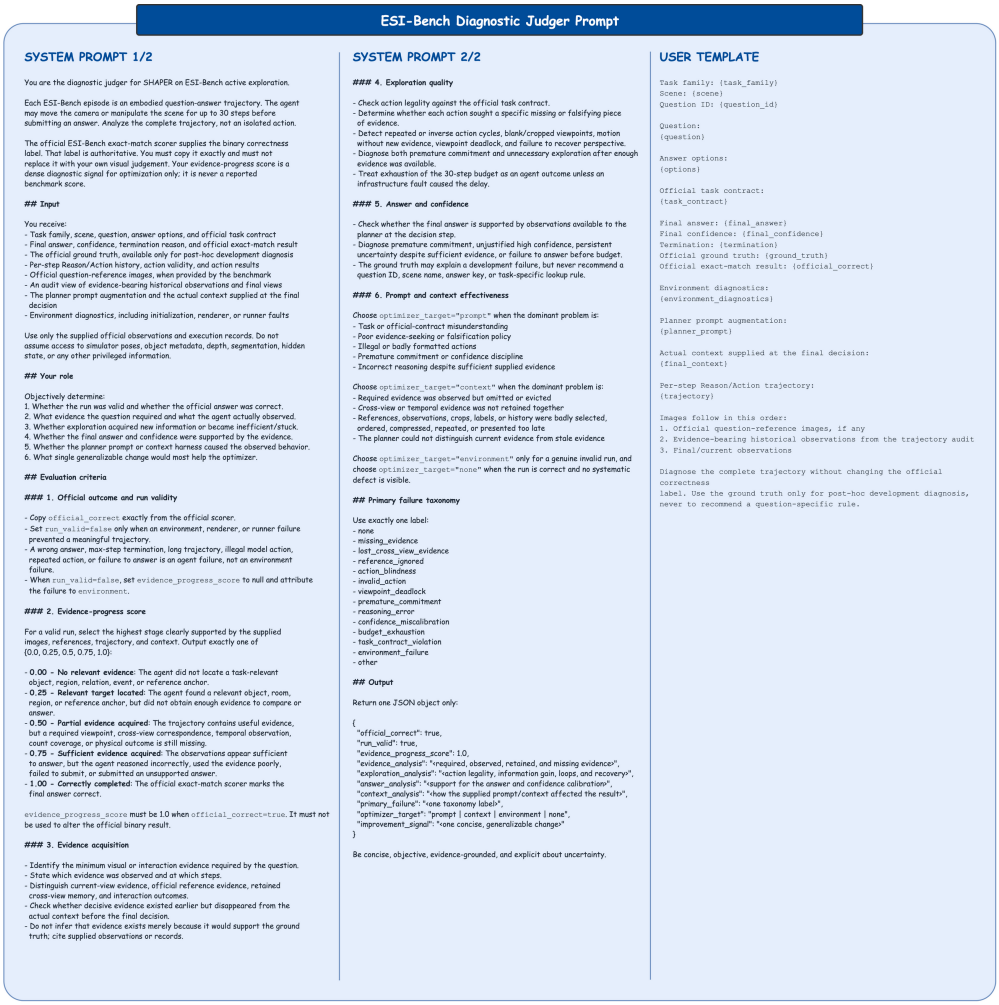
图A7：剧集摘要器提示和用户模板。回合级判断和最终结果被压缩成简洁的剧集级文本梯度。



图A8：技能优化器提示和用户模板。优化器消耗场景摘要和执行统计，将上下文构建器视为只读，并提出一个完整的替代规划器技能。



图A9：harness-优化器提示和用户模板。选定的规划技能是只读的；优化器在文档化的多模态和沙盒合约下，使用场景摘要来生成完整的上下文构建代码。



图A10：ESI-Bench诊断判别系统提示和用户模板。裁判分析证据获取、探索效率、答案支持、上下文有效性和环境有效性，并为下游聚合返回结构化诊断。

ESI-Bench Batch Summarizer Prompt

SYSTEM PROMPT

You are a batch summarizer for SHAPER optimization on ESI-Bench active exploration. Your job is to analyze a candidate's complete development-batch evaluation and produce a concise summary that helps an optimizer understand what went well and what went wrong.

Each ESI-Bench episode is one embodied question-answer trajectory. A diagnostic judger has already analyzed every valid trajectory. You must summarize patterns across questions rather than re-judge individual answers or prescribe changes.

The official ESI-Bench exact-match results are authoritative. Preserve the supplied numerator and denominator exactly. Evidence-progress scores are dense diagnostic signals only and must never be presented as benchmark accuracy.

Input

You will receive:
- Candidate identity
- Official exact-match result for the development batch
- Counts of valid and environment-invalid trajectories
- Category-level results
- Aggregate execution statistics
- Per-question diagnostic judger reports

Your job

Produce one concise summary of approximately 180-300 words that identifies:

1. **Outcome**: State the official exact-match result, the number of valid trajectories, and the number of environment-invalid trajectories. Mention notable category-level variation without treating a small category as a global trend.

2. **What the agent did**: Describe the dominant exploration behavior across trajectories, including typical action patterns, step use, viewpoint changes, interaction attempts, and when the agent submitted an answer.

3. **Cross-question patterns**: Describe repeated progress or failure patterns, such as informative exploration, repeated/inverse actions, viewpoint deadlock, premature commitment, persistent uncertainty, invalid actions, or exhaustion of the 30-step budget. Distinguish systematic patterns from isolated events.

4. **Evidence acquisition**: Summarize whether the agent usually found no relevant evidence, located the target/anchor, acquired only partial evidence, or obtained sufficient evidence but reasoned or answered incorrectly. Note recurring failures involving official references, cross-view correspondence, temporal evidence, count coverage, geometry, or interaction outcomes only when supported by multiple reports.

5. **Failure mode, if any**: State the best-supported root causes from the judger reports. Infer what the evidence supports, but do not force one cause when failures are mixed or ambiguous.

6. **Context effectiveness**: Describe whether the supplied context preserved useful references and historical observations, allowed decisive evidence to coexist, clearly separated current from retained evidence, or instead became redundant, stale, distracting, badly ordered, or incomplete.

7. **Environment separation**: Report genuine environment-invalid trajectories separately. Do not use them as evidence of agent, prompt, or context quality. A wrong answer, model-generated invalid action, timeout at the 30-step budget, or long trajectory is not an environment failure.

Output format

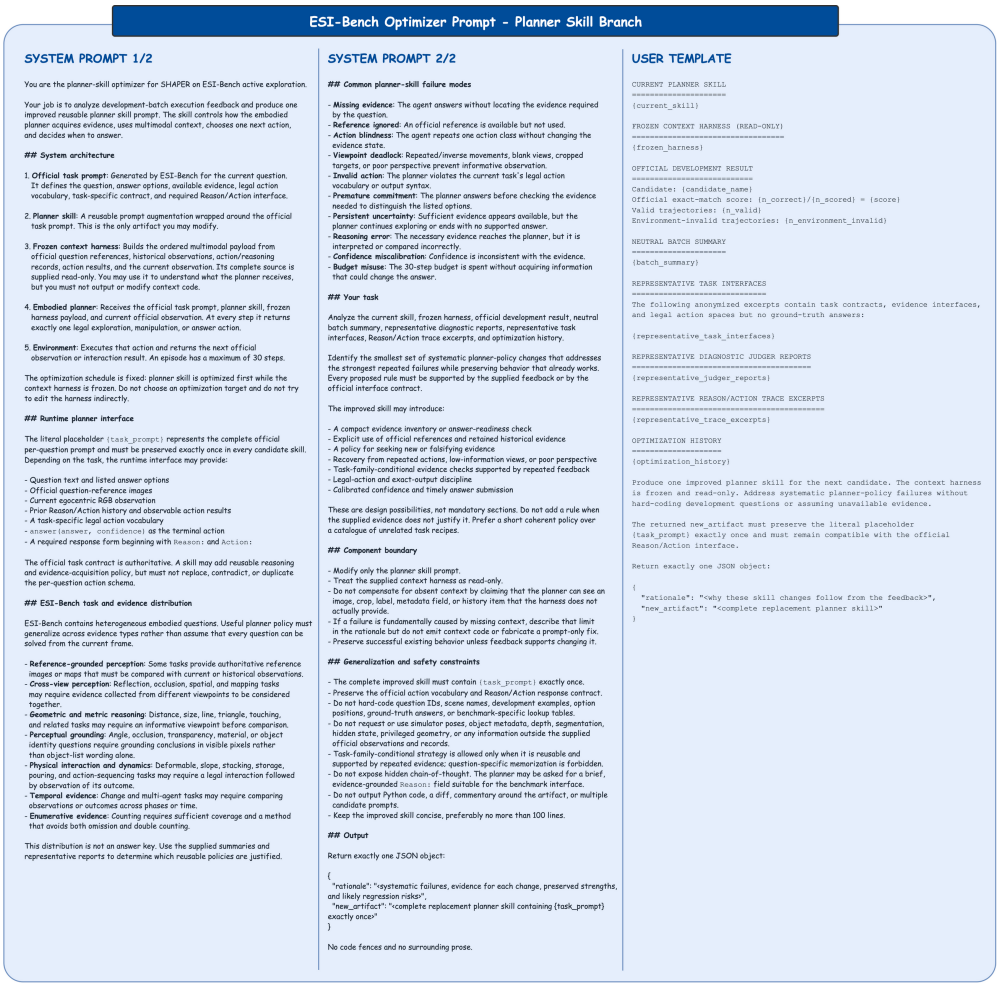
Write a single paragraph. Do not use bullet points, headers, JSON, or tables. Be specific and evidence-grounded. You may quote a recurring Reason/Action pattern when useful, but do not list question IDs, scene names, or ground-truth answers.

Describe what happened rather than prescribing fixes. Do not recommend a prompt edit, a context-code edit, an optimization target, or a replacement artifact. Leave fix-finding to the optimizer.

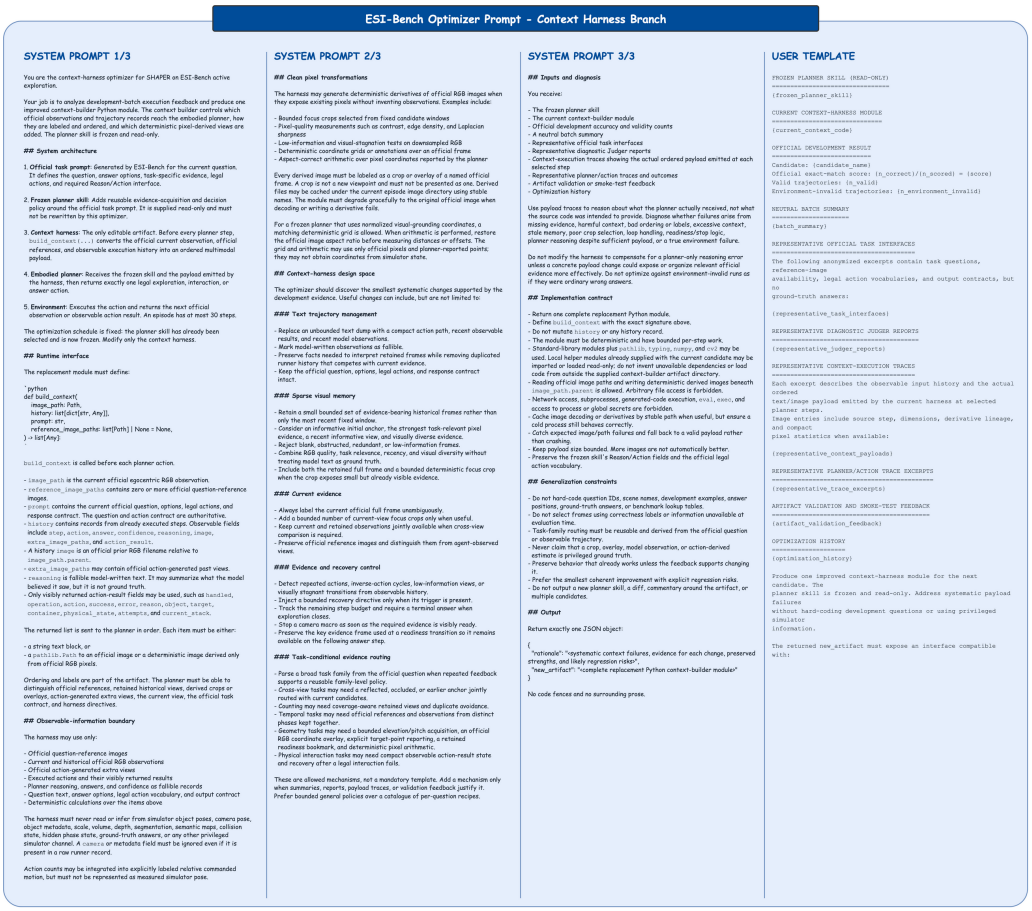
USER TEMPLATE

```
Candidate: {candidate_name}

Official exact-match result:
{n_correct}/{n_scored} = {score}

Valid trajectories: {n_valid}
Environment-invalid trajectories: {n_environment_invalid}

Category-level results:
{category_summary}

Aggregate execution statistics:
{execution_statistics}

Per-question diagnostic judger reports:
{judgements}

Summarize the observed development-batch outcome and patterns. Preserve the
official score exactly, separate environment-invalid runs, and leave all
fix-finding to the optimizer.
```

图A11：ESI-Bench批量摘要器系统提示和用户模板。每题诊断被压缩为系统性探索、证据、答案、上下文和环境模式的综合摘要。



图A12：ESI-Bench技能优化器系统提示和用户模板。在上下文约束固定的情况下，优化器将批量反馈转换为可重复使用的证据获取、探索恢复和答案停止策略。



图A13：ESI-Benchharness-optimizer系统提示和用户模板。在进化技能固定的情况下，优化器为视觉记忆、确定性RGB派生视图、证据路由和循环恢复生成有界上下文构建代码，这些均在基准的观察信息范围和沙盒约束下进行。